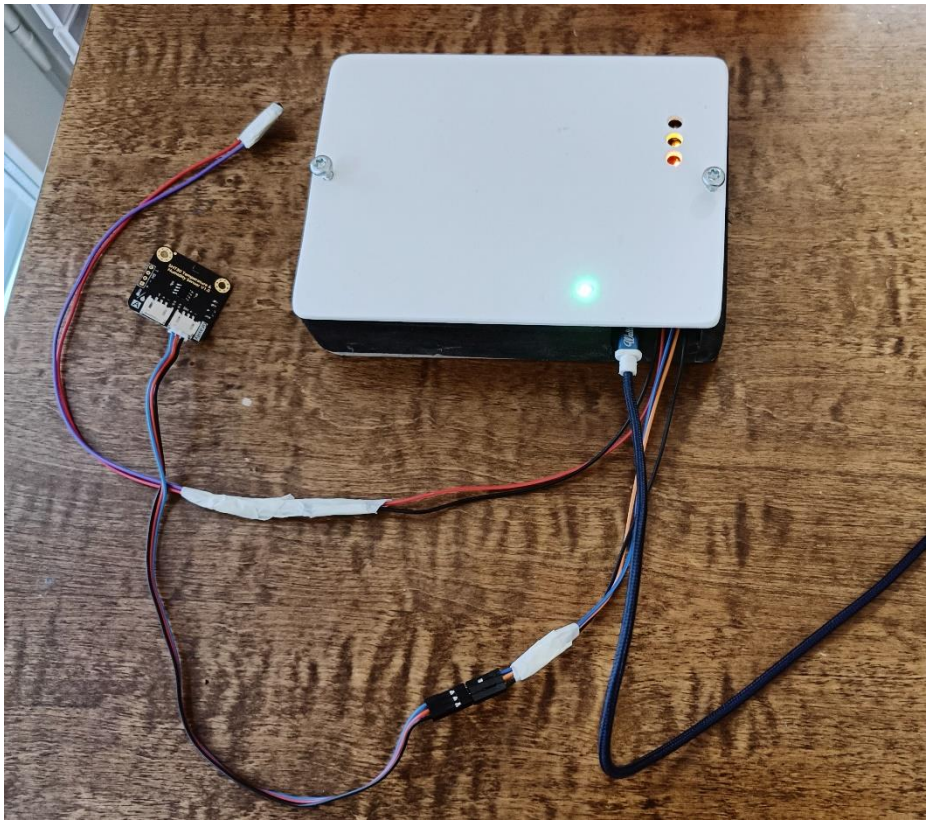
Juha Laurila

Projekti 3-kurssi

Kosteuden ja lämpötilan valvontajärjestelmä kiinteistöön

°C RH



Insinööri

Tieto- ja viestintätekniikka

Syksy 2023

KAMK • University of Applied Sciences

Sisällys

1 Johdanto 1

2 Projektin vaiheet ja ajankäyttösuunnitelma 2

3 Toteutus 4

3.1 Alustava ohjelma ja sen kytkennät 4
3.2 Anturit, sovituskytkennän suunnittelu ja simulointi Tina TI:llä 6
3.2.1 Anturit 6
3.2.2 Yleistä sovituskytkennöistä 6
3.2.3 Sovituskytkentä LM335:lle 6
3.2.4 Sovituskytkentä HIH-4000:lle 11
3.3 Sovituskytkentäkortin toteutus 19
3.4 Bluetooth-toiminnon käyttöönotto 22
3.5 Anturien liittäminen ja testaus 25
3.6 Lopullinen ohjelma, kotelointi ja paristokäyttö 30

4 Yhteenveto 33

5 Liitteet 34

5.1 Alustavan testiohjelman koodi 34
5.2 Lopullinen ohjelmakoodi 39

1 Johdanto

Projektin tavoitteena oli suunnitella ja toteuttaa yksinkertainen kiinteistön valvontajärjestelmä, jossa oli lämpötilan ja kosteuden mittaus sekä niiden seuranta myös etänä. Kehitysalustana ja järjestelmän keskusyksikkönä oli Arduino Nano 33 IoT -mikrokontrollerikehitysalusta. Lämpötila- ja kosteusanturina käytettiin analogisia antureita, joiden kytkeminen Arduino Nano 33 IoT:n AD-muuntimelle vaati sovituskytkennän suunnittelun ja toteutuksen.

Järjestelmän ohjelmointi tapahtui Arduino IDE -kehitysympäristössä. Projektiin liittyi lisäksi etäkäytön toteuttaminen puhelimella tai työpöytäkäyttöliittymällä, joko Bluetoothilla tai WiFiä käyttäen. Arduino Nano 33 IoT:ssa ovat molemmat toiminnot sisäänrakennettuna. Lopullisessa projektin määrittelyssä etäkäyttö sovittiin toteutettavaksi BLE-toiminnolla (Bluetooth Low Energy). Keskusyksikölle - Arduino Nanon ja sovituskytkentäelektroniikan muodostamalle kokonaisuudelle tuli suunnitella myös kotelointi, jossa tuli olla lisäksi akun/paristojen käyttömahdollisuus.

2 Projektin vaiheet ja ajankäyttösuunnitelma

Aiemmat Arduino-harjoitukset ja analogiaelektroniikan suunnittelukurssit antoivat varsin hyvän pohjan projektin suunnittelulle. Jo projektin määrittelyvaiheessa tuli aika nopeasti käsitys, miten missäkin osa-alueessa tulisi pääpiirteissään edetä ja paljonko se vaatisi suurin piirtein aikaa. Tarkkaa ajankäyttösuunnitelmaa en kuitenkaan laatinut. Projekti ja sen ajankäyttö jakaantuisi kuitenkin muutamaan selkeään kokonaisuuteen.

Ensimmäisenä oli alustavan perusohjelman ja demokytkennän toteuttaminen. Tämä nopeahko vaihe meni jotakuinkin rutiinilla, mutta auttoi hyvin kertaamaan asioita ja orientoimaan projektin ohjelmointi-, kytkentä- ja muihin haasteisiin. Samalla pääsin myös toteamaan jatkokehitystä ajatellen, että ohjelma, kytkentä ja Arduino Nano-alusta toimivat yhdessä oikein.

Seuraavaksi oli vuorossa varsinaisiin antureihin ja niiden ominaisuuksiin tutustuminen sekä sovituskytkentöjen laskenta ja suunnittelu Arduino Nanon AD-muuntimelle. Tähän vaiheeseen liittyi myös suunnitellun kytkennän simulointi Tina TI -ohjelmaa käyttäen. Tälle kokonaisuudelle varasin mielessäni reilummin aikaa, koska tuli uusia haastavampia asioita ja tietoa piti etsiä erilaisilta datalehdiltä ja aiemmilta kursseilta.

Kolmantena oli sovituskytkentöjen käytännön toteutus – komponenttien hankkiminen ja piirilevyn toteutus. Vaikka olin aiemmin tutustunut Eagle -piirilevyn suunnitteluohjelmaan, ajattelin toteuttaa sovituskytkentäpiirin kuitenkin täysin itse juottaen, kun se oli mahdollista ja kyse oli vain yhdestä piirilevystä. Rajasinkin projektista piirilevyn suunnittelemiseen ja ulkopuolisella teettämiseen liittyvät ylimääräiset toimenpiteet ajansäästösyistä pois, varsinkaan kun se ei ollut päätarkoitus projektissa. Ajattelin käyttäväni reilusti aikaa ja ajatusta tähänkin vaiheeseen.

Neljäs osa-alue oli ohjelman Bluetooth-ominaisuuksien opiskelu, testaus ja implementointi lopulliseen ohjelmaan. BLE oli vielä jäänyt aiemmilta kursseilta varsin pinnallisen tiedon varaan. Pääasiallisena tiedonlähteenä tulisi olemaan Beginning Arduino Nano IoT -kirja, jossa oli myös muutama perusharjoitus erilaisista BLE-sovelluksista. Tähänkään aihepiiriin syventymisen en ajatellut myöskään tapahtuvan kovin vähässä ajassa.

Viidentenä oli kokonaisuuden testaus sovituskytkentää käyttäen, aluksi potentiometrin avulla ja myöhemmin varsinaisia antureita käyttäen. Kun järjestelmä saatiin toimimaan riittävän hyvin koekytkentäalustalla ja ohjelma oli hiottu riittävän valmiiksi, oli järjestelmän lopullisen

toteuttamisen vuoro. Kokonaisuudesta piti saada kompakti, akku/paristotoiminen ja siististi koteloitu kokonaisuus, lisättynä paikallisilla merkkivaloledeillä.

3 Toteutus

3.1 Alustava testiohjelma ja sen kytkennät

Aluksi päätin tehdä yksinkertaisen testiohjelman, jossa sytytetään ja sammutetaan LEDejä. Ensin oli vuorossa Arduino Nano 33 IoT:n käyttöönotto, jossa aiemmin jo mainitusta Beginning Arduino Nano IoT -kirjasta oli iso apu. Kun sain ohjelmointiympäristön tarvittavine kirjastoineen asennettua, käännettyä ja ajettua ohjelman onnistuneesti Arduino Nano:lle, oli aika ruveta jalostaman ohjelmaa.

Liitin mukaan ensimmäisen potentiometrin, jonka tarkoitus oli simuloida anturin antamaa jännitettä. Potentiometrin reunimmaiset johtimet kytkin Nano:n 3,3 V- ja Gnd-nastaan sekä potentiometrin keskimmäisen johtimen yhteen Nanon analogiatulonastoista (A0). Tämä potentiometrin asennosta riippuva potentiometrin analoginen output-jännite menee näin Nanon 10-bittiselle AD-muuntimelle, jossa se muunnetaan digitaaliseksi tiedoksi. Tarkemmin ilmaistuna 10-bittisen AD-muuntimen resoluutio (erottelukyky) on 2 potenssiin 10 eli tässä tapauksessa muuntimelle menevä jännitealue 0-3,3 V voi saada yhteensä 1024 eri arvoa (0-1023).

Hyödynsin sitten koodissa erilaisissa kaavoissa mitattavan suureen arvon näyttämiseksi kyseistä AD-muuntimen antamaa bittisanaa – digitaalista arvoa, joka saatiin A0-analogiatulon kautta potentiometrilta. Tein kaksi kaavaa, toisen lämpötilalukemaa ja toisen jännitelukemaa varten. Periaate on seuraava: ensin luetaan analogiatulon (A0) kautta tullut digitaaliseksi muunnettu arvo kaavassa käytettävään muuttujaan: *sensorValue0 = analogRead(A0);*

Kokonaislukumuuttujaa (*sensorValue0*) , johon on sijoitettu AD-muuntimelta saatu arvo, hyödynnetään esim. seuraavassa lämpötilanmittauskaavassa (mittausalue -40 - +50 °C), jonka tulos taas sijoitetaan liukulukumuuttujaan *T*: *T = ((90.0 / 1023.0)*sensorValue0)-40 ;*

Vastaavasti, tein jännitelukeman näyttämiseen kaavan: *V=(3.3 / 1023.0)*sensorValue0 ;*
Saatuja arvoja voitiin sitten käyttää esim. Arduinon sarjamonitorissa, sarjaplotterissa tai ulkoisessa näytössä: *Serial.print("Lämpötila on: "), Serial.print(T);* (sarjamonitoriin).

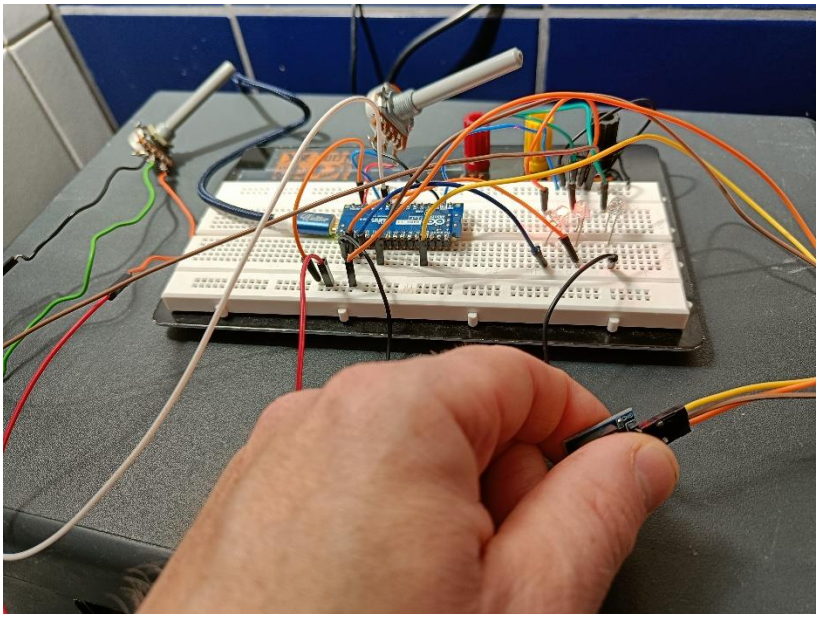
Lisäsin myös toisen potentiometrin analogiatulo (A2): een, suhteellisen kosteuden (RH) mittaamiseksi. Simuloin sitten erilaisia mittaustilanteita – lämpötilan ja suhteellisen kosteuden vaihteluita. Muokkasin ohjelmaan ehtorakenteita ja tein kytkentäalustalle ledeillä erilaisia kytkentöjä

kuvaamaan erilaisia lämpötilojen ja suhteellisen kosteuden olosuhteita.
Otin testiohjelmassa käyttöön myös painonapin, jolla vaihdettiin ohjelman suoritus ajonaikaisesti lämpötilamittauksesta suhteellisen kosteuden mittaukseen. Painonapin käyttöönotto koodissa tapahtui seuraavasti:

int pushButton = 7; // painonappi-muuttujan esittely ja alustus
int state = 0; // painonapin tila -muuttujan esittely ja alustus
pinMode(pushButton, INPUT); // painonapin asetus (pin 7) inputiksi
state = digitalRead(pushButton); // luetaan painonapin tila (joko 0 =ei painettu tai 1=painettu)

Painonapin tilaa käytetään sitten ehtolauseissa hyväksi:

if (state == 1){ … koodi… } // jos painonapin tila = 1 = painetaan nappia -> kosteusmittaukseen,
else { … koodi… } // jos painonapin tila = 0 (ei paineta nappia) -> lämpötilan mittaukseen.



Kuva 1. Testikytkentä, jossa kaksi potentiometriä, ledit sekä painonappi.

3.2 Anturit, sovituskytkentöjen suunnittelu ja simulointi Tina TI:llä

3.2.1 Anturit

Projektin määrittelyssä oli sovittu käytettäväksi analogisia LM335 -lämpötila- ja HIH-4000 kosteusanturia. LM335-anturi oli sovituskytkennän suunnittelun ja käyttöjännitteensä kannalta ongelmaton Nanon kanssa käytettäväksi. HIH-4000:n suhteen oli anturin käyttöjänniteasia epävarma, saadaanko lopullista anturikytkentää liitettyä mitenkään Arduino Nanon IoT:n 3,3 V:n järjestelmään. Se ei kuitenkaan estänyt tekemästä sovituskytkentää projektin vaatimukset täyttävällä tavalla, enkä antanut sen asian enempää alkuvaiheessa häiritä. Vaihtoehtoinen kehitysalusta olisi ollut 5 V jännitteellä toimiva Arduino MKR, jossa HIH-4000 olisi toiminut ongelmitta.

3.2.2 Yleistä sovituskytkennöistä

Aloitin sovituskytkentöjen suunnittelun palauttamalla mieleen Analogiaelektroniikan suunnittelu -kurssilla opittua ja tutkimalla tehtyjä laskelmia. Totesin että on käytettävä ns. case 2 -tyyppistä siirtofunktion laskuyhtälöä anturijännitteiden muuttamiseksi AD-muuntimelle sopivaksi sovituskytkentäpiirin avulla. Sovituskytkennässä käytettäisiin, kuten aiemmissakin harjoitustöissä tarkoitukseen sopivaa TLV2372 operaatiovahvistinta ja negatiivisella takaisinkytkennällä olevaa vahvistinkytkentää, johon tulisi mitoittaa tarvittavat vastukset halutun sovituskytkentätuloksen aikaansaamiseksi. Eli kun anturilta saadaan tietty jännitealue, on se sovitettava AD-muuntimelle siten, että jännitteet ovat halutulla mittausalueella 0-3,3V. Näin saadaan AD-muuntimen koko resoluutio käyttöön ja mittauksesta mahdollisimman tarkka.

3.2.3 Sovituskytkentä LM335:lle

LM 335:n lähtöjännitteitä ei saada anturin datalehdeltä suoraan, vaan ne täytyy laskea datalehdellä ilmoitetun kalibrointijännitteen ja jännitteenmuutoskertoimen mukaan (jännitteen muutos / K).

Lähtöjännitteiden laskeminen:
Mittausalue -40 C - +50 C

Kalibrointilämpötila T0 = 25 °C .

(Kalibrointilämpötila kelvineinä= 273K+25K = 298 K)

Kalibrointijännite 2,98 V

Jännitteenmuutoskerroin 10 mV / K

Lähtöjännite (T -40) = 2,98V - ((40 K + 25 K) *0,01 V)

Lähtöjännite (T -40) = 2,98V - (65 K * 0,01 V) =2,98V - 0,65V

Lähtöjännite (T -40) =2,33 V

Lähtöjännite (T 50) = 2,98V + (25 K *0,01V) = 2,98V + 0,25 V

Lähtöjännite (T 50) =3,23 V

Lähtöjännitteiden taulukointi:

Anturin lähtöjännite:	Sovituskytkennän jännite (AD-muuntimelle):
2,33 V	0 V
3,23 V	3,3 V

Siirtofunktion laskeminen: Käytetään Case 2 -kytkentää: Vo = Vin*m + b

Yhtälöt:

0 = 2,33m + b

3,3 = 3,23m + b

1. Ratkaistaan alemmasta yhtälöstä m:

3,3 = 3,23m + b =>

3,23m = 3,3 - b I : 3,23

=> $m = \frac{3,3-b}{3,23}$

2. Sijoitetaan m:n lauseke ylempään yhtälöön:

$0 = 2{,}33\frac{(3{,}3-b)}{3{,}23}$ + b I *3,23

0=2,33(3,3-b) + 3,23b

0=7,689 -2,33b + 3,23b

0=7,689 + 0,87b

-0,87b=7,689 I : -0,87

b= -8,8379

3. Sijoitetaan b:n arvo alempaan yhtälöön

3,3 = 3,23m -8,8379

3,23m = 3,3+8,8379

3,23m = 12,139 I :3,23

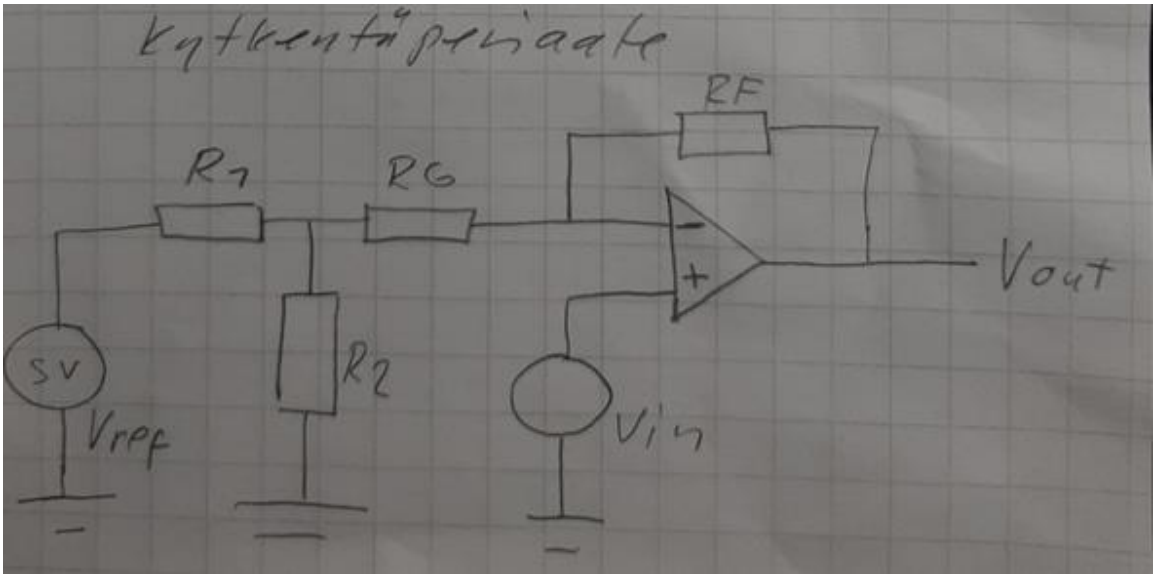
m=3,7582

Siirtofunktio on:

b = -8,8379 ja m: 3,7582

Vout = Vin * m + b

Vout = Vin * 3,7582 - 8,8379



Kuva 2. LM335-anturin sovituskytkennän periaate

$$Vout = Vin\frac{(RF+RG+R1\ II\ R2)}{RG+R1\ II\ R2} - Vref\frac{(R2)}{R1+R2} \quad \frac{(RF)}{RG+R1\ II\ R2}$$

m I b I

koska R1 II R2 << RG

-> $m = 3{,}7582 = \frac{RF+RG}{RG}$

-> $m = \frac{2{,}7582+1}{1}$

-> RF = 2,7582RG

jos RG-vastuksen arvo = 20 kohm

-> RF = 2,7582 * 20 kohm

-> RF-vastuksen arvo = 55,164 kohm

-> $I\ b\ I = Vcc\frac{(RF)}{RG}\ \frac{(R2)}{R1+R2}$ I Vcc = Vref 3,3 V

-> $8{,}8379 = 3{,}3\ V\frac{55{,}164\ kohm}{20\ kohm} * \frac{(R2)}{R1+R2}$ I * 20 kohm

-> $175{,}758 = \frac{182{,}0412R2}{R1+R2}$ I * R1 + R2

175,758 R1 + 175,758 R2 = <u>182,0412 R2</u>

175,758 R1 = 182,0412 R2 - 175,758 R2

175,758 R1 =6,2832 I : 175,758

R1-vastuksen arvo = 0,03575 kohm

alussa on ehto R1 II R2 << (RG) 20 kohm

eli 2 ehtoa: R2 > R1 (R1=0,03575R2)

R1 II R2 << 20 kohm

-> kokeilu: jos R2-vastuksen arvo = 1kohm

-> R1 = 0,03575R2* 1 kohm =35,75 ohm

-> R1 + R2 = 35,75 +1 1,03575

-> 1,03575 kohm << RG

Vastusarvot sovituskytkentään:

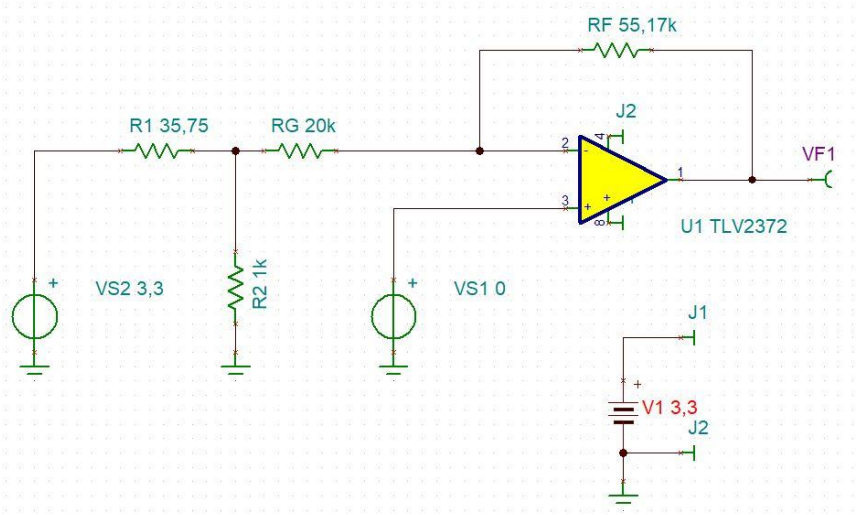
R1 = 35,75 ohm

R2 = 1 kohm

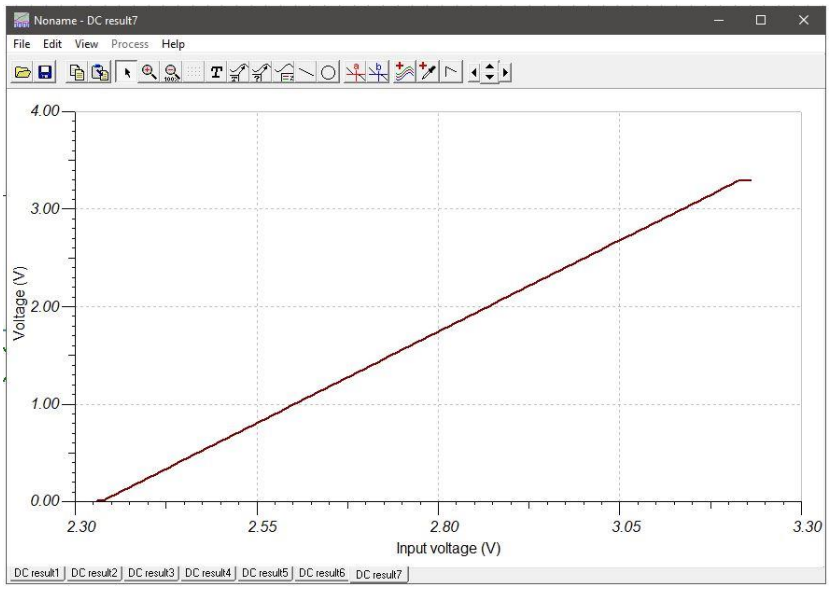
RF = 55,164 kohm

RG = 20 kohm

Lopullinen kytkentä:



Kuva 3. LM335:n sovituskytkennän simulointi TinaTI: llä.



Kuva 4. LM335:n sovituskytkennän simulointianalyysi TinaTI:llä.

3.2.4 Sovituskytkentä HIH-4000:lle

HIH-4000 anturin sovituskytkennän tein sillä oletuksella, että se saataisiin toimimaan Arduino Nano 33:n kanssa. Käytännössä käyttöjännite oli ongelmana; kyseinen anturi kun vaatii min. 4 V käyttöjännitteen, jolloin Nanon 3,3 V:lla ko. anturi ei luonnollisestikaan suoraan toimisi vaan vaatisi korkeampaa ulkopuolista jännitettä. (Myöhemmin, kun anturin varsinaista liittämistä kokeilin, osoittautui, että johtuen vaadittavasta kytkentätavasta anturia ei voi liittää kyseiseen alustaan). Suunnittelin ja toteutin sitä ennen kuitenkin sovituskytkennän, huomioimatta lopullista anturin liitettävyyttä ja käyttöjänniteongelmaa.

HIH-4000:n tapauksessa oli myös käytettävä case 2 -tyyppistä siirtofunktion laskuyhtälöä. Anturin lähtöjännitteet oli anturin datalehdeltä kerrottu suoraan, niitä ei tarvinnut erikseen laskea.

V (0 % RH) = 0,8 V

V (100 % RH) = 3,9 V

Lähtötietojen alustava taulukointi:

Anturin lähtöjännite:	Sovituskytkennän jännite AD-muuntimelle:
0,8 V	0 V
3,9 V	3,3 V

Mutta koska Arduino Nano 33:n referenssijännite on 3,3 V, piti anturin antama max. 3,9 V jännite RH 100 %: ssa ensin vaimentaa max. 3,3 V:ksi, jotta sovituskytkentä toimi oikein.

Vaimennusta varten piti laskea vaimennuskerroin ja toteuttaa vaimennus jännitteenjakokytkennän avulla.

Anturin lähtöjännitteen vaimennuskertoimen laskeminen:

Sovituskytkennän Vmax / anturin lähtö Vmax
-> 3,3V / 3,9V = n. 0,8461538 (vaimennuskerroin)

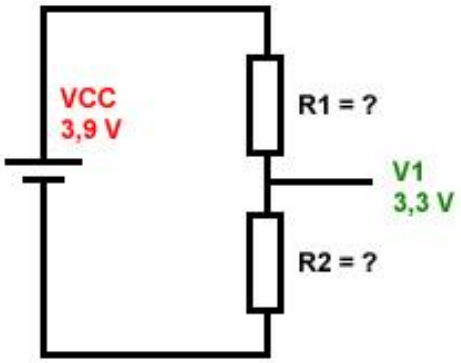
Anturin lähtöjännitteet vaimennuksen jälkeen:

V (0 % RH) = 0,8V*0,8461538 = 0,6769 V -> 0,68 V
V (100 % RH) = 3,9V*0,8461538 = 3,2999 V -> 3,3 V

Koska vaimennuskerroin on 0,8461538, on anturin lähtöjännitteet kerrottu 0,8461538:lla ja saatu seuraavat jännitteet, joita voi käyttää sovituskytkennässä:

Anturin lähtöjännite vaimennuksen jälkeen:	Sovituskytkennän jännite AD-muuntimelle:
0,68 V (RH 0%)	0 V
3,3 V (RH 100 %)	3,3 V

Vaimennuksen jännitteenjakokytkentä:
Lasketaan max RH 100 % = 3,9 V (anturin lähtöjännite) -> 3,3 V (vaimennettu jännite) mukaan.



Kuva 5. Lähtötilanne HIH-4000:n jännitteenjakokytkennälle

Jos R2 = 1kohm

➔ $R1 = \left(R2 * \frac{VCC}{V1}\right) - R2$

➔ $R1 = \left(1\ kohm * \frac{3{,}9\ V}{3{,}3\ V}\right) - 1\ kohm$

➔ $R1 = \left(\frac{3{,}9\ kohmV}{3{,}3\ V}\right) - 1\ kohm$

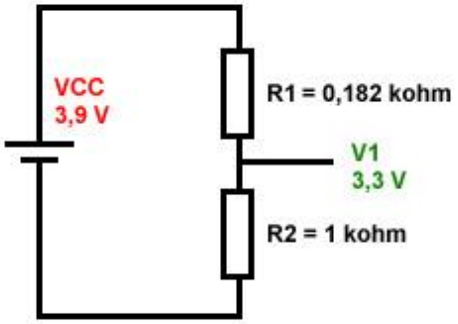
➔ 1,181818181 kohm – 1 kohm

= 0,181818181 kohm = 182 ohm

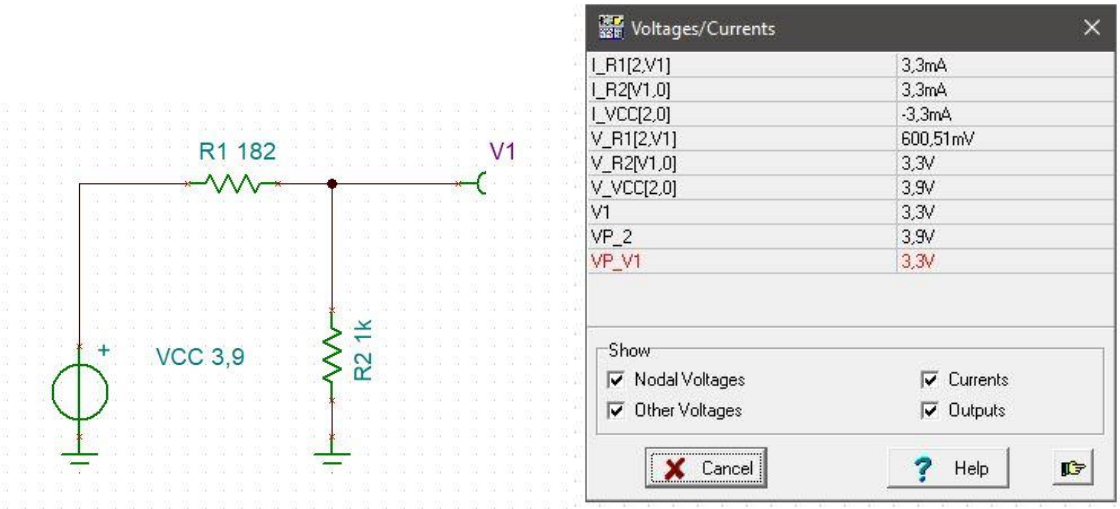
Eli jännitteenjakokytkennässä

R1 = 182 ohm

R2 = 1 kohm



Kuva 6. HIH-4000:n jännitejako ja ratkaistut vastusarvot:



Kuva 7: HIH-4000:n jännitejaon TinaTI -simulointi

Varsinaisen siirtofunktion laskeminen:

Käytetään ns. case 2 -kytkentää: Vout = Vin*m + b

Yhtälöt:

0 = 0,68m + b

3,3 = 3,3m + b

1. Ratkaistaan ensin alemmasta yhtälöstä m:

3,3 = 3,3m + b =>

3,3m = 3,3 – b I : 3,3 =>

-> $m = \frac{3{,}3-b}{3{,}3}$

2. Sijoitetaan sitten m:n lauseke ylempään yhtälöön:

-> $0 = 0{,}68\frac{(3{,}3-b)}{3{,}3}$ + b I * 3,3

0=0,68 (3,3-b) +3,3b

0=2,244-0,68b + 3,3b

0=2,244 + 2,62b

2,62b = -2,244 I : 2,62

b= -2,244:2,62

b=-0,8565

3. Sijoitetaan b:n arvo alempaan yhtälöön

3,3 = 3,3m + b

3,3 = 3,3m + (-0,8565)

3,3 = 3,3m-0,8565

3,3m = 3,3+0,8565

3,3m = 4,1565 I : 3,3

m = 4,1565 : 3,3

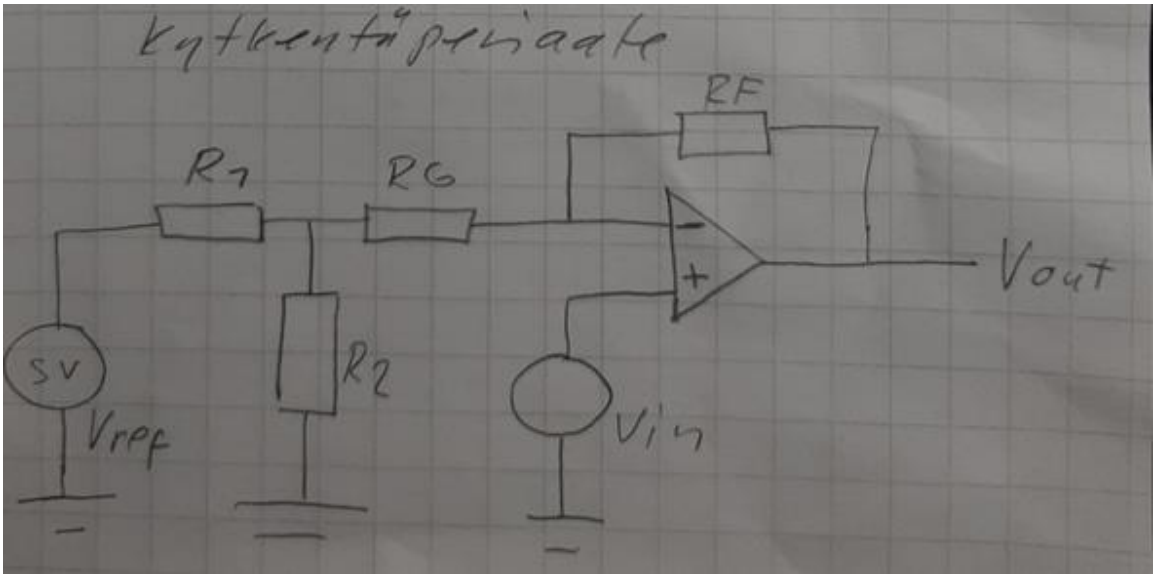
m= 1,2595

Siirtofunktio on:

Vout = Vin*m + b

b = -0,8565 ja m: 1,2595

Vout = Vin*1,2595 - 0,8565



Kuva 8: HIH-4000-anturin sovituskytkennän periaate

$$Vout = Vin\frac{(RF+RG+R1\ II\ R2)}{RG+R1\ II\ R2} - Vref\frac{(R2)}{R1+R2} \quad \frac{(RF)}{RG+R1\ II\ R2}$$

m I b I

koska R1 II R2 << RG

->$m = 1{,}2595 = \frac{RF+RG}{RG}$

->$m = \frac{0{,}2595+1}{1}$

RF = 0, 2595RG

jos vastus RG = 20 kohm

-> RF = 0, 2595*20 kohm

-> vastus RF = 5,19 kohm

-> $I\ b\ I = Vcc\frac{(RF)}{RG} \quad \frac{(R2)}{R1+R2}$ I Vcc = Vref 3,3 V

-> $0{,}8565 = 3{,}3\ V * \frac{5{,}19\ kohm}{20\ kohm} * \frac{(R2)}{R1+R2}$ I * 20 kohm

-> $17{,}13 = \frac{17{,}127R2}{R1+R2}$ I * R1 + R2

17,13 R1 + 17,13 R2 = 17,127 R2

17,13 R1 =17,127 R2 - 17,13 R2

17,13 R1 =-0,17362R2 I : 17,13

R1=0,0100992R2

alussa on ehto R1 II R2 << (RG) 20 kohm

eli 2 ehtoa: R2> R1 (R1=0,0100992R2)

R1 II R2 << 20 kohm

kokeilu: jos vastus R2 = 1kohm

-> vastus R1 = 0,0100992* 1 kohm = 0,0100992kohm = 10,09 ohm

R1 II R2 = 0,0100992

R1 + R2 = 0,0100992+1 = 1,0100992 kohm

->1,0100992 kohm << RG

Vastusarvot sovituskytkentään:

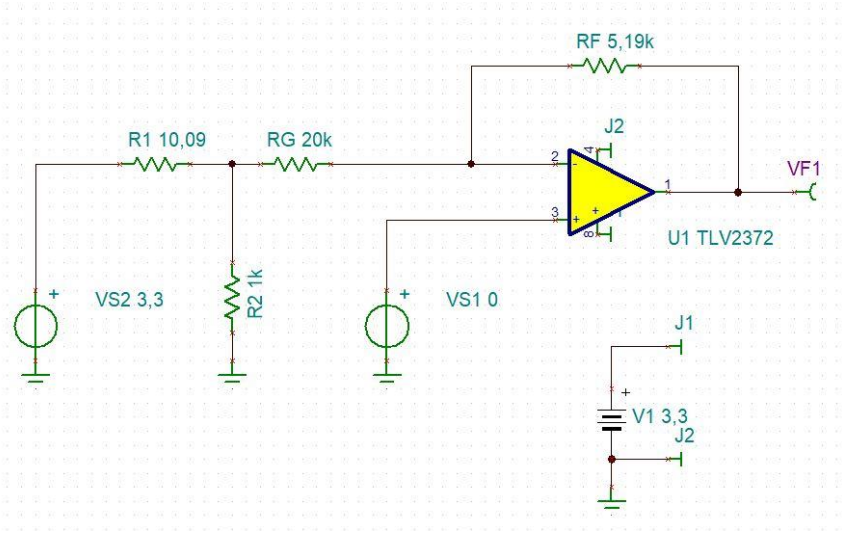
RF=5,19 kohm

RG=20 kohm

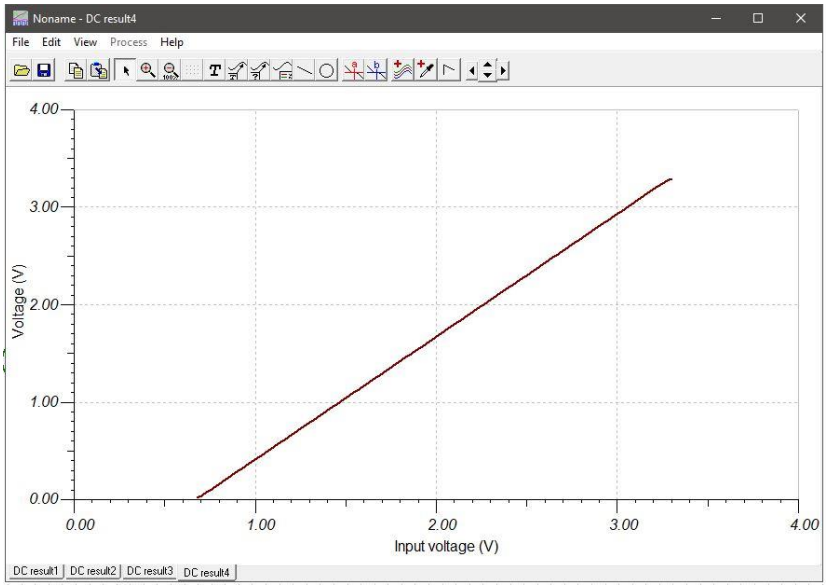
R2 = 1 kohm

R1 = 10,09 ohm

Lopullinen kytkentä:



Kuva 9. HIH-4000:n sovituskytkennän simulointi TinaTI:llä.



Kuva 10. HIH-4000:n simulointianalyysi TinaTI:llä.

3.3 Sovituskytkentäkortin toteutus

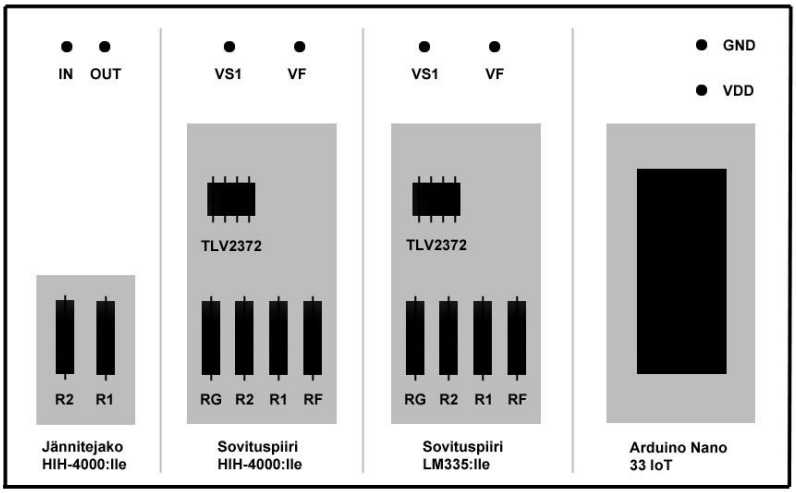
Kun sovituskytkentä oli suunniteltu ja testattu TinaTI:lla, oli aika toteuttaa sovituskytkentä käytännössä. Minulla oli jo molemmat anturit sekä sovituskytkennän operaatiovahvistinkomponentit hankittuna, joten puuttui vain sopiva juotosalusta komponenteille sekä vastukset. Vastuksien valinnasta ei ollut paljon kokemusta, mutta siinä auttoi tarkan vastusluettelon laatiminen ja vastusten värikoodeihin perehtyminen. Koululta valitsin sitten sopivat vastukset ja reikälevyn kytkentäpiirille. Vastuksien valinta oli oma operaationsa, kun moni laskettu vastusarvo piti yhdistellä useammasta vastuksesta.

Ennen kuin aloitin piirilevyn tekoa, tein vastusten kanssa mittausoperaation yleismittarin kanssa. Näin varmistuin vielä vastusarvoista sekä laitoin jokaisen kytkennän vastukset nimettyihin pusseihin järjestykseen. Tämä auttoi suuresti varsinaisen juotostyön tekemisessä. Lisäksi tuli harjoiteltua vähälle jäänyttä yleismittarin käyttöä.

Reikälevylle piti siis mahtua:

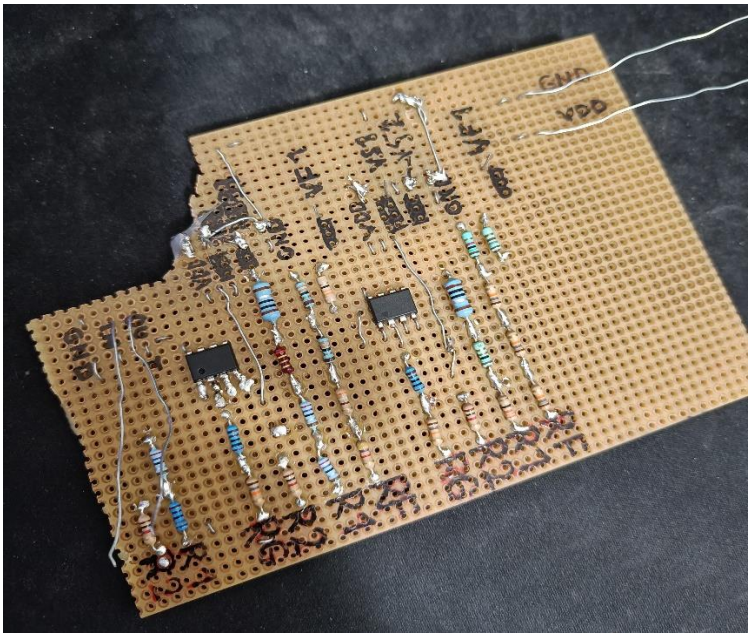
1. Jännitejakokytkentä HIH-4000-anturin jännitteen vaimennukselle
2. HIH-4000-anturin sovituskytkentäpiiri
3. LM335-anturin sovituskytkentäpiiri
4. Tila johdotuksille ja liitynnöille
5. Tila 9 V paristolle ja Arduino Nano 33 IoT:lle

Valitsin varsin suurikokoisen reikälevyn juottamiseen, mikä oli hyvä päätös.
Mikäli piiri olisi suunniteltu esim. Eaglella ja toteutettu koneellisesti olisi asiat luonnollisesti mahtuneet reilusti pienempään tilaan. Vaikka en tehnyt millintarkkaa layout-suunnitelmaa ennen juottamista, suunnittelin levyn tilankäytön ja kytkentöjen tilankäytön mahdollisimman systemaattiseksi ja tarkoituksenmukaiseksi.



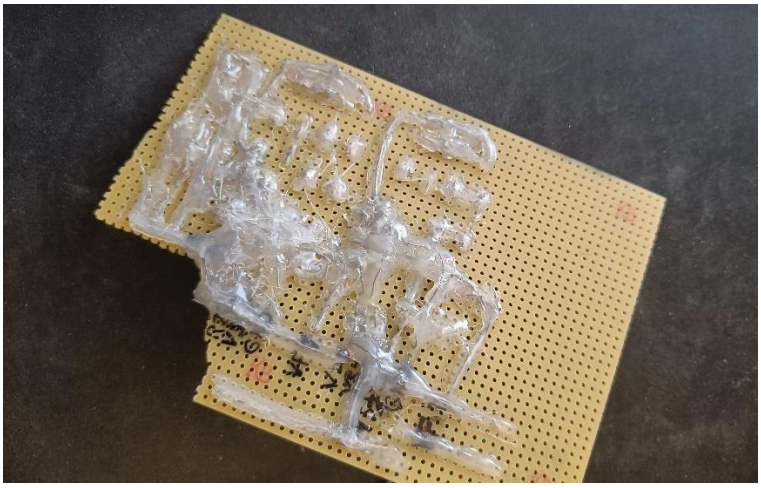
Kuva 11. Piirilevyn periaatteellinen layout ilman johdotuksia.

Paristokotelon sijainti on piirilevyn toisella puolella esim. Arduino Nanon alapuolella.



Kuva 12. Juotettu, keskeneräinen sovitinkytkennän piirilevy (yläpuoli).

Käytännössä koululta piirilevy ei ollut ihan täydellinen suorakaide, vaan siitä oli palanen murrettu pois HIH-4000 jännitteenjaon ylänurkasta, toteutus mukaili kuitenkin yo. layoutia. Koska kaikkia vastusarvoja ei ollut valmiina, piti osa niistä tehdä yhdistämällä eriarvoisia vastuksia.



Kuva 13. Juotettu, keskeneräinen sovitinkytkennän piirilevy (alapuoli).

Johdotukset tein ohuella teräslangalla, joista valtaosa on piirilevyn alapuolella, eristettynä kuumaliimalla toisistaan. Juotoksista ja johdotuksista ei tullut kovin siistejä, osin huonon kolvin takia, mutta keskityin toimivuuteen ja tarkoituksenmukaisuuteen.

Kun komponentit oli juotettu ja johdotukset eristetty, mittasin vastusarvot kortilta yleismittarilla ja vertasin niitä laskennallisiin, TinaTI: llä simuloituihin arvoihin. Lisäksi testasin kortin toimivuuden potentiometrillä ja ohjelmalla.

LM335 lämpötila-anturi

1. LM335.n sovituskytkennästä mitatut vastusarvot:

R1 = 35,8 ohm (laskennallinen 35,75 kohm) = 100,14 % laskennallisesta

R2 = 0,984 kohm (laskennallinen 1 kohm) = 98,4 % laskennallisesta

RF = 54,5 kohm (laskennallinen 55,17 kohm) = 98,7 % laskennallisesta

RG = 19,73 kohm (laskennallinen 20 kohm) = 98,65 % laskennallisesta

2. LM335:n sovituskytkennän testaus potentiometrillä ja ohjelmalla:

potentiometrin mitattu jännite: 2,32 V -> ohjelma (AD-muuntimelta): 0,01 V

potentiometrin mitattu jännite: 2,77 V -> ohjelma (AD-muuntimelta): 1,5 V

potentiometrin mitattu jännite: 0,68 V -> ohjelma (AD-muuntimelta): 3,3 V

HIH4000 kosteusanturi

1. HIH4000:n jännitejakokytkennästä mitatut vastusarvot:

R1 = 182,4 ohm (laskennallinen 182 ohm) = 100,1 % laskennallisesta
R2 = 0,993 kohm (laskennallinen 1 kohm) = 99,3 % laskennallisesta

2. LM4000: n sovituskytkennästä mitatut vastusarvot:

RF=5,13 kohm (laskennallinen 5,19 kohm) = 98,8 % laskennallisesta
RG=19,7 kohm (laskennallinen 20 kohm) = 98,5% laskennallisesta
R2 = 984 ohm (laskennallinen 1 kohm) = 98,4 % laskennallisesta
R1 = 9,1 ohm (laskennallinen 10,09 kohm) = 90,2 % laskennallisesta

Testasin vielä mitatuilla oikeilla vastusarvoilla TinaTI:ssä, oli OK.

3. HIH4000:n sovituskytkennän testaus potentiometrillä ja ohjelmalla:

potentiometrin mitattu jännite: 0,67 V -> ohjelma (AD-muuntimelta): 0,02 V
potentiometrin mitattu jännite: 3,3 V -> ohjelma (AD-muuntimelta): 3,3 V

3.4 Bluetooth-toiminnon käyttöönotto

Järjestelmään piti vielä liittää BLE-toiminnallisuus, jotta lämpötila- ja mittaustiedon lukeminen onnistuisi esim. puhelimella. Lähdin opettelemaan asiaa tekemällä Beginning Arduino Nano IoT -kirjan Bluetooth-harjoituksia. Harjoitukset olivat selkeitä ja opettivat Bluetoothin käyttöönottoa ja erilaisten asioiden toteuttamista askel askeleelta. Latasin puhelimeeni kirjan suositteleman nRF Connect -sovelluksen, jonka avulla kirjan mukainen etäohjaus onnistui varsin sujuvasti.

Kun riittävällä tasolla ymmärsin BLE:n käyttöönotto- ja toimintarutiinit kooditasolla sekä totesin niiden toimivuuden käytännössä, oli aika ottaa ne käyttöön mittaussovelluksessani.

Aluksi koodissa luodaan Bluetooth-palvelu ja sille UUID-tunniste:
BLEService measureService("16150f38-e7a9-4fe1-ae08-48464baf25b2");

Pitkä merkkisarja on UUID, (Universally Unique Identifier, yksilöllinen identifiointitunniste), jota käytetään mm. BLE-sovelluksissa tunnistamaan luotu toiminto. Tunniste voidaan luoda esim. seuraavalla generaattorilla: *https://www.guidgenerator.com/online-guid-generator.aspx.*

Bluetooth-palvelulla voi olla tiettyjä ominaisuuksia, kuten esim. lämpötilanmittaus:

BLEStringCharacteristic temperatureLevel("ff99948c-18ff-4ed8-942e-512b9b24b6db", BLERead | BLEWrite | BLENotify,15); jolla voi olla luku (*BLERead)*, kirjoitus (*BLEWrite)* ja jatkuva / automaattinen tiedon syöttäminen etälaitteelle (*BLENotify)* esim. lämpötilan muuttuessa.

Bluetooth alustetaan kutsumalla funktiota:
BLE.begin()

Asetetaan palvelun paikallinen nimi ja itse palvelu mainostettavaksi:

BLE.setLocalName("Measurements");
BLE.setAdvertisedService(measureService);

Asetetaan BLE-palvelun ominaisuudet:
measureService.addCharacteristic(temperatureLevel);

Lisätään BLE-palvelu:
BLE.addService(measureService);

Asetetaan alkuarvot ominaisuuksille:
temperatureLevel.writeValue(String(0));

Aloitetaan BLE-palvelun mainostaminen:
BLE.advertise();

Asetetaan keskuslaite (Nano) kuuntelemaan yhteydenottoja
BLEDevice central = BLE.central();

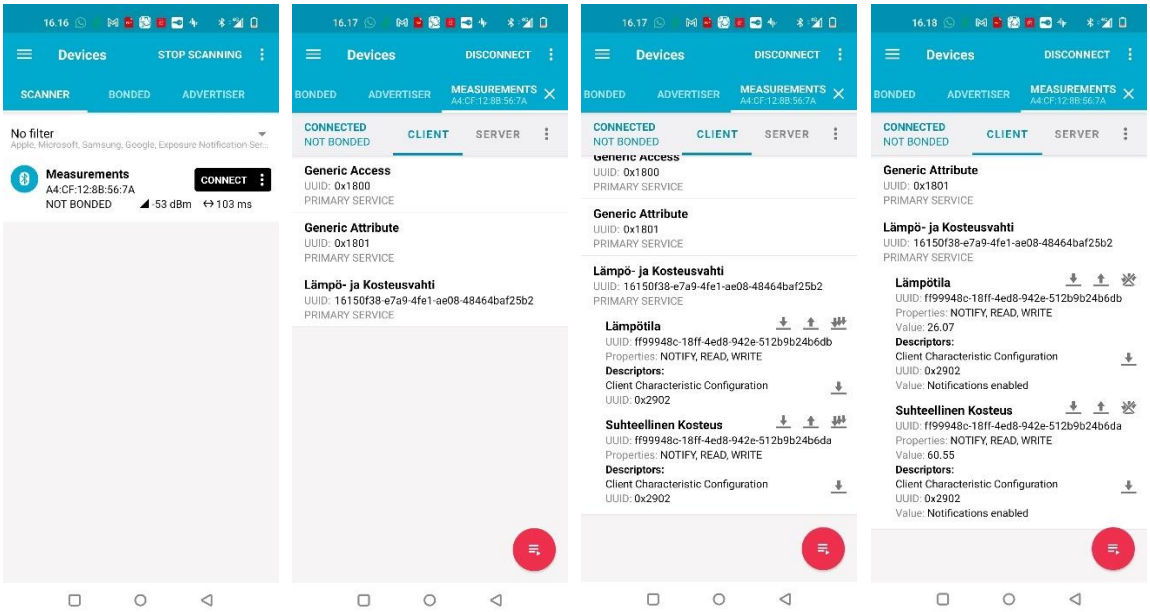
Kun yhteys keskuslaitteelle on muodostunut, suoritetaan esim. koodit…

```
if (central) {

Serial.print("Connected to central: "); // printataan sarjamonitoriin viesti lukijan liittymisestä
Serial.println(central.address()); // kirjoitetaan keskuslaitteen osoite sarjamonitoriin

// kun BLE-keskuslaite on liittyneenä etälaitteeseen...

 while (central.connected()) {
 digitalWrite(ledPin, HIGH);
 updateMeasureLevel();  // kutsutaan esim. funktiota, jolla luetaan mittaustieto
 delay(1000);
  }
```

Kun yhteys etälaitteelta katkaistaan:

```
digitalWrite(ledPin, LOW);  // sammutetaan led
Serial.print("Disconnected from central: "); // kirjoitetaan viesti sarjamonitoriin
Serial.println(central.address()); // kirjoitetaan keskuslaitteen osoite sarjamonitoriin

}
```

Android-puhelimellani toimiva nRF Connect -sovellus oli ihan kelvollinen näyttämään mittaustietoja ja sillä pystyi myös kontrolloimaan järjestelmää. Toki jos etäkäytöstä olisi tehnyt monipuolisemman ja käytettävyydeltään fiksumman, olisi voinut kehittää oman helppokäyttöisen sovelluksensa sitä varten. Tietoa olisi voinut esittää erilaisina ajan suhteen muuttuvina käyrinä, siihen olisi voinut liittää erilaisia laskureita, tallennustoimintoja jne.



Kuva 14. Kuvakaappaukset nRF Connect -sovelluksesta, jolla luetaan BLE:n välityksellä mittausta. Kun painetaan ”kolme nuolta” -kuvaketta, saadaan Notify-tiedot mittausjärjestelmältä (lämpötilan ja kosteuden mittauksen jatkuvasti ja automaattisesti päivittyvät tiedot). Yksi nuoli alaspäin on yhden luennan Read-tieto ja yksi nuoli ylöspäin Write-tieto, jolla voitaisiin ohjata järjestelmän toimintaa.

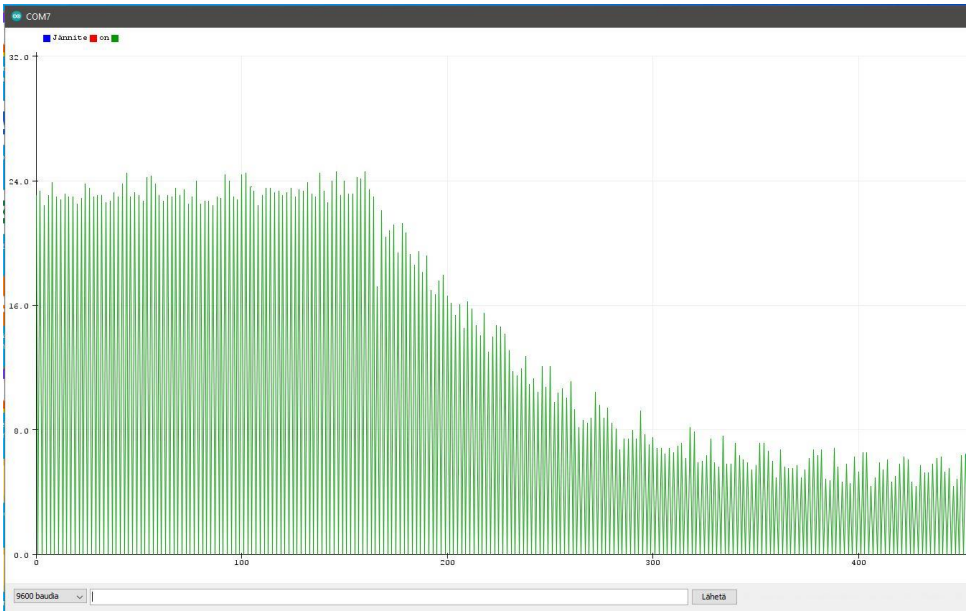
3.5 Anturien liittäminen ja testaus

Sovituskytkentäpiirilevyn testattuani aloin liittämään siihen antureita. Aloitin LM335-lämpötila-anturista. Koululta saamassani valmiissa anturikortissa oli kolme erilaista anturia, LM335, HIH-400 sekä jokin valoisuusanturi. Oma aikansa meni, kun tutkin korttia, johtimia ja antureiden pinnejä selvittääkseni, mitkä pinnit tarkoittavat mitäkin ja miten kytkeminen tapahtuisi. LM335-anturissa on kolme pinniä: GND, output ja adjust. GND kytketään normaalisti sovituskytkennän/Nanon GND-nastaan, output kytketään sovituskytkennän 3,3 V:een sekä vastuksen kautta Nanon ko. analogiatuloon. Irrotin anturin kortista saadakseni kytkettyä sen halutusti. Anturikortissa vastusarvo oli 2,2 kohm. Testasin sillä ensin, mutta en saanut sitä sillä toimimaan. Sain selvitettyä että 3,3 V käyttöjännitteellä se pitää olla pienempi. Muutin sen 300 ohmiksi.

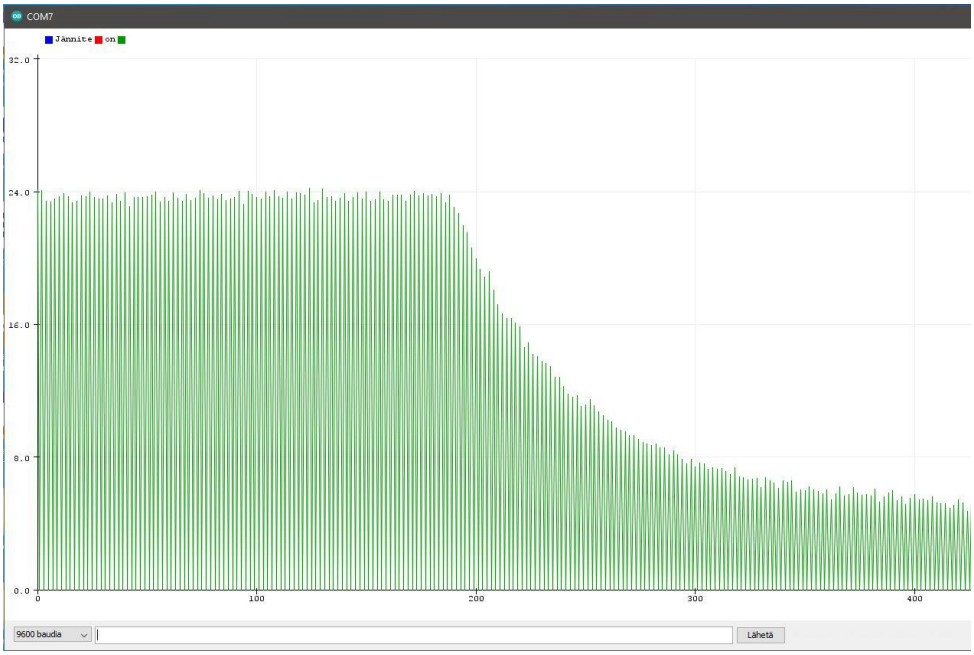
Anturin adjust-pinni oli anturikortissakin kytkemättä mihinkään, mitä aikani ihmettelen. Se onkin tarvittaessa tehtävää kalibrointia varten ja jätettiin omassakin kytkennässä vapaaksi. En kuitenkaan saanut vieläkään anturia toimimaan halutusti. Tässä vaiheessa sain opettajalta

vertailukytkennän opettajalta, josa R2-vastus oli muutettu 3 kohmiksi. Nyt anturi toimi auttavasti ja reagoi muutoksiin hyvin, joskin lukemat vaihtelivat huomattavasti mittausten välillä (serial plotterin tulostus huomattavan kohinainen).

Tein mittauksille koodissa keskiarvoistuksen, jolloin kohinakin rauhoittui.
Käytännössä mittasin 8 arvoa pienin viivein eri muuttujiin ja laskin sitten niiden keskiarvon, joka sijoitettiin omaan muuttujaansa, joka tulostettiin sarjamonitoriin sekä etälaitteen käyttöön.



Kuva 15. LM335:n Serial plotter -tulostuskäyrä ilman keskiarvoistusta. Anturilta tuleva jännite ja siitä laskettava mittaustieto vaihtelee varsin paljon (=kohinaa).



Kuva 16. LM335:n Serial plotter -tulostuskäyrä koodissa tehdyn keskiarvoistuksen jälkeen. Mittauskäyrä on huomattavasti tasaisempi.

Mittasin LM335:llä pakastimen lämpötilan (asetettu -18.een), kehonlämmön kyynärtaipeesta sekä maksimilämmön varovasti lämmittämällä anturia.

```
Lämpötila on: -14.78
Jännite on: 0.84
Suhteellinen kosteusproentti on: 63.10
Jännite on: 1.79
Lämpötila on: -15.17
Jännite on: 0.85
Suhteellinen kosteusproentti on: 62.83
Jännite on: 1.81
Lämpötila on: -14.92
Jännite on: 0.89
Suhteellinen kosteusproentti on: 65.65
Jännite on: 1.79
Lämpötila on: -14.86
Jännite on: 0.89
Suhteellinen kosteusproentti on: 62.96
Jännite on: 1.83
Lämpötila on: -15.17
Jännite on: 0.83
Suhteellinen kosteusproentti on: 63.10
Jännite on: 1.81
Lämpötila on: -14.97
Jännite on: 0.84
Suhteellinen kosteusproentti on: 62.70
Jännite on: 1.81
Lämpötila on: -14.78
Jännite on: 0.82
Suhteellinen kosteusproentti on: 62.96
Jännite on: 1.81
Lämpötila on: -15.03
Jännite on: 0.87
Suhteellinen kosteusproentti on: 62.56
Jännite on: 1.81
```

Kuva 17. Pakastimesta mitatut lämpötilat (-18:een asetettu)

```
Lämpötila on: 37.70
Jännite on: 2.64
Suhteellinen kosteusproentti on: 62.16
Jännite on: 1.80
Lämpötila on: 37.49
Jännite on: 2.62
Suhteellinen kosteusproentti on: 63.10
Jännite on: 1.81
Lämpötila on: 37.67
Jännite on: 2.64
Suhteellinen kosteusproentti on: 61.49
Jännite on: 1.80
Lämpötila on: 37.78
Jännite on: 2.64
Suhteellinen kosteusproentti on: 62.70
Jännite on: 1.81
Lämpötila on: 37.79
Jännite on: 2.64
Suhteellinen kosteusproentti on: 62.70
Jännite on: 1.80
Lämpötila on: 38.47
Jännite on: 2.69
Suhteellinen kosteusproentti on: 62.96
Jännite on: 1.80
Lämpötila on: 37.68
Jännite on: 2.63
Suhteellinen kosteusproentti on: 63.23
Jännite on: 1.79
Lämpötila on: 37.92
Jännite on: 2.64
Suhteellinen kosteusproentti on: 62.16
Jännite on: 1.80
```

Vieritys Show timestamp

Kuva 18. Kyynärtaipeesta mitatut lämpötilat

COM7

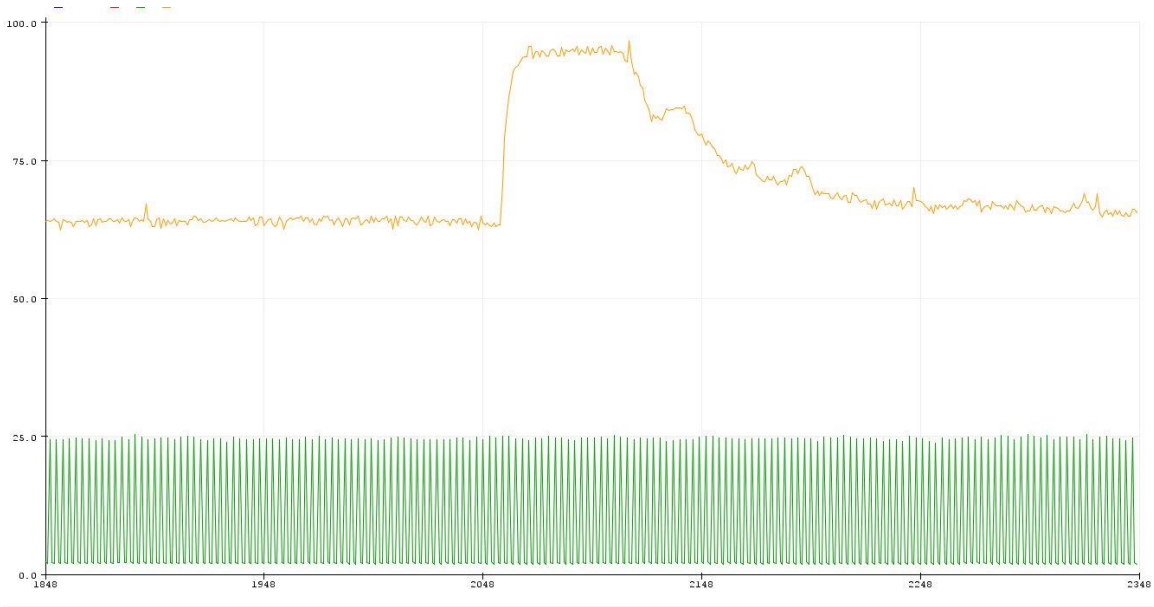
```
Lämpötila on: 48.80
Jännite on: 3.04
Suhteellinen kosteusproentti on: 55.85
Jännite on: 1.64
Lämpötila on: 48.96
Jännite on: 3.01
Suhteellinen kosteusproentti on: 55.85
Jännite on: 1.64
Lämpötila on: 48.85
Jännite on: 3.03
Suhteellinen kosteusproentti on: 55.18
Jännite on: 1.66
Lämpötila on: 48.92
Jännite on: 3.03
Suhteellinen kosteusproentti on: 55.44
Jännite on: 1.61
Lämpötila on: 48.87
Jännite on: 3.04
Suhteellinen kosteusproentti on: 55.85
Jännite on: 1.64
Lämpötila on: 48.98
Jännite on: 3.04
Suhteellinen kosteusproentti on: 53.97
Jännite on: 1.60
Lämpötila on: 49.00
Jännite on: 3.03
Suhteellinen kosteusproentti on: 55.98
Jännite on: 1.64
Lämpötila on: 49.15
Jännite on: 3.03
Suhteellinen kosteusproentti on: 55.31
Jännite on: 1.64
```

Vieritys Show timestamp

Kuva 18. Maksimilämpötilat anturia lämmittämällä.

Pakastimesta ja kehonlämmön mittauksen perusteella lukemat ovat siis vajaat 2 astetta liian korkeat. Toisaalta maksimilämpö, minkä järjestelmä antaa, on liian alhainen, pitäisi olla n. 57 astetta.

Kosteusanturi HIH-4000 ei siis toiminut 3,3V järjestelmässä, (min supply 4 V), tilalle sain SHT30 anturin, jossa oli mukana sekä lämpötilan, että kosteudenmittaus. Ko. anturin lämpötilanmittausta en tietenkään projektissa tarvinnut. Anturi toimi 3,3 V:llä ja ei vaatinut sovituskytkentää (tehty anturiin sisäänrakennetusti). Tutkin anturin tietoja ja tein koodin sovituslaskelman ohjeen mukaisesti. Ko. anturilla sain RH:n menemään 95 %:een (märkä paperi), huonekosketus oli n 3% päässä hiuslanka-kosteusmittarin näyttämästä ja anturi reagoi ihan ok muutenkin muutoksiin. Sovituskytkentäkortti jäi siis HIH-400-anturin osalta käyttämättä Arduino Nanolle soveltumattoman käyttöjännitteen takia, vaikka toimikin testatusti potentiometrillä ja yleismittarilla jännitteet mitaten. Toisin sanoen, kaikkien lenkkien pitää toimia, jotta kokonaisuus toimisi.

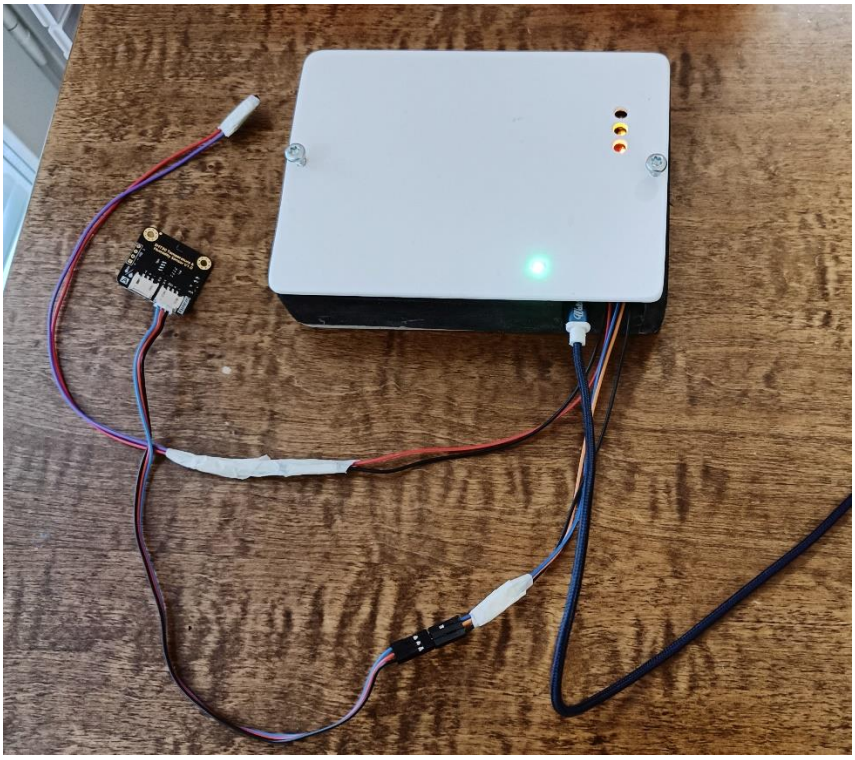


Kuva 19. Kosteusanturin tulostuskäyrä serial plotterissa. Korkea RH-lukema keskellä saavutettiin käyttämällä märkää paperia anturia vasten. Muuten arvot vastaavat huoneilman kosteutta. Huonelämpötila on n. 25 astetta.

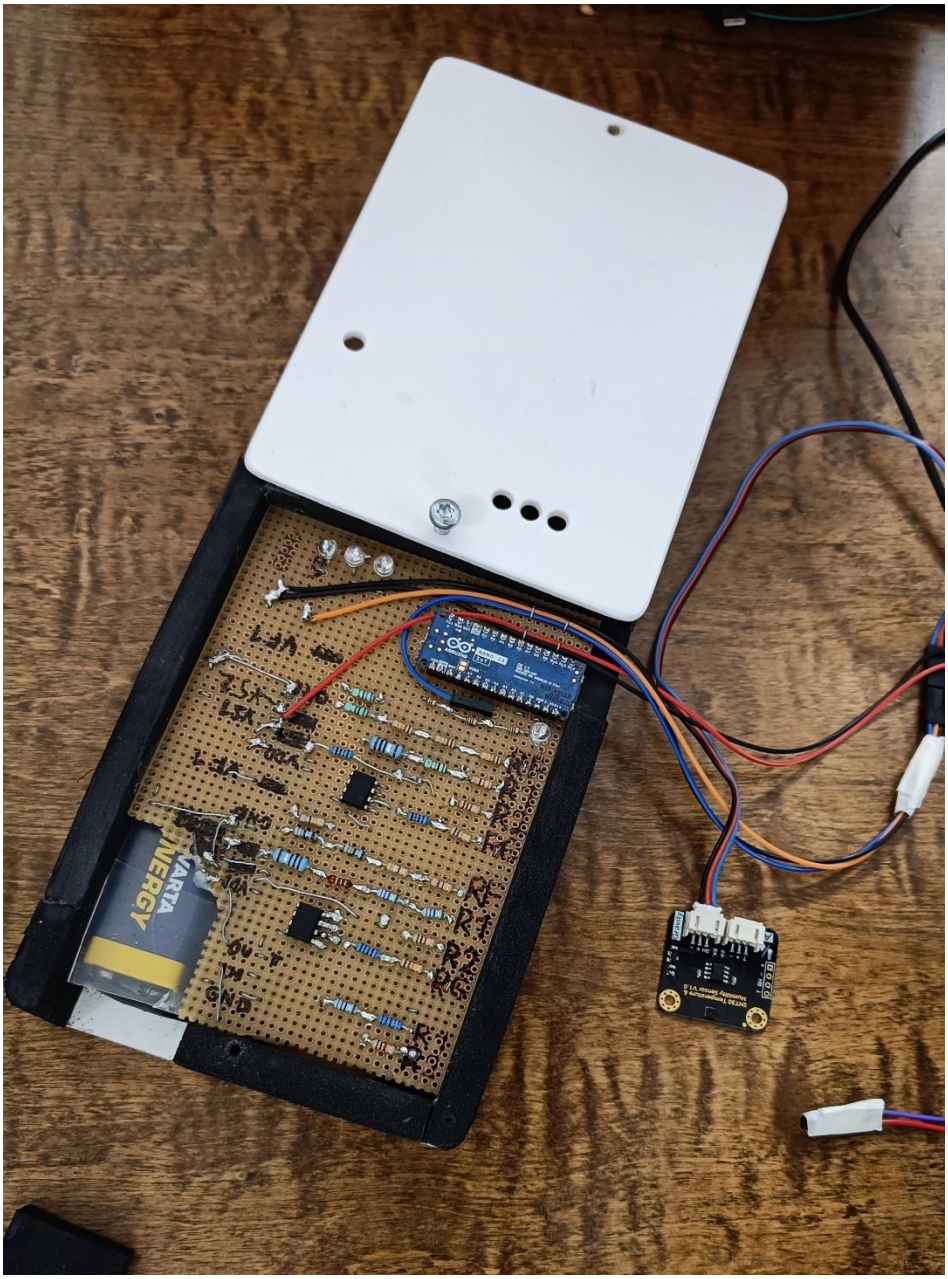
3.5 Lopullinen ohjelma, kotelointi ja paristokäyttö

Täydensin vielä piirilevyä lämpötilaa indikoivilla kolmella led-valolla (sininen, keltainen ja punainen) sekä kosteutta indikoivalla vihreällä ledillä. Vihreä led on kytketty Nanon sisäänrakennettuun lediin (pin13), joten se välkkyy myös silloin, kun ohjelmaa ajetaan kortille. Koodit ledeille tein yksinkertaisilla ehtolauseilla, jotka vaihtavat ledien väriä lämpötilan mukaan sekä muuttavat vihreän ledin vilkkumista kosteuden mukaan. Lopuksi tein piirilevykokonaisuudelle kotelon. Koska sopivaa aihiota ei ollut saatavilla, rakensin kotelon kahdesta valkoisesta HD-levystä sekä kovasta solukumilevystä leikkaamalla ja liimaamalla. Porasin reiät ledeille, hioin kansilevyn siistimmäksi ja maalasin spraylla. Kiinnityksen tein kahdella ruuvilla.

Paristokäyttöä varten tein keskusyksikön runkoon tilan 9 V paristolle, joka kiinnitetään navoistaan sisältä tulevien johtimien päässä olevaan liittimeen. USB-portilla käytettäessä Nano 33 IoTeen tulee 5 V jännite ja liitettäessä ulkoinen tulojännite Vin-nastaan on alueen oltava välillä 4,5 -21 V. Nanon sisäinen jännitteensäädin muuttaa joka tapauksessa käyttöjännitteen sitten 3,3V:ksi.



Kuva 20. Järjestelmä toiminnassa, käyttöjännite USB-johdon kautta.



Kuva 21. Keskusyksikkö kansi aukaistuna.

4 Yhteenveto

Projekti oli mielenkiintoinen ja monipuolinen. Opin paljon uusia asioita antureista, sovituskytkennän suunnittelusta, ohjelmoinnista, mittauksista, piirilevyn suunnittelusta ja toteutuksesta sekä Bluetooth-ominaisuuksien käyttöönotosta. Projektin kautta sainkin hyvän kokonaiskäsityksen niistä työvaiheista, mitä vastaavantyyppinen projekti pitää pääpiirteissään sisällään – suunnittelusta valmiiksi tuoteprototyypiksi.

Vaikka toteutus käsittää toiminnoiltaan vain lämpötilan ja kosteusmittauksen, antoi se hyvin haastetta suunnitteluun ja toteutukseen. Järjestelmää on helppo laajentaa liittämällä keskusyksikköön esim. lcd-näyttö sekä myös muita mittauksia ja toimintoja. Myös Arduino Nano IoT:n omien anturien tietoa, esim. asennon tai tärinän tunnistamista (kiintyvyysanturi) on helppo ottaa käyttöön. Voitaisiin myös ottaa mittausjärjestelmään käyttöön pilvipalvelu ja lukea sieltä selaimella netin yli mistä päin maailmaa tahansa.
Lisäksi voitaisiin kehittää räätälöity fiksumpi sovellus esim. mobiililaitteelle, jolla järjestelmää operoitaisiin, varustettuna grafiikoilla, muistilla ja kehittyneemmällä ominaisuuksilla.

Projekti antoi käsitystä siitäkin, että vaikka laskennat ja simuloinnit ovat oikea lähtökohta suunnittelulle, voi todellisia komponentteja joutua käytännössä vielä säätämään, jotta asiat toimivat myös käytännössä juuri suunnitellulla tavalla. Jatkuva lisätiedon haku esim. datalehdiltä ja tehdyistä toteutuksista netistä on oleellista uuden tiedon hankinnassa ja omaksumisessa. Kokemusta karttui ja kynnys madaltui suunnitella myös myöhemminkin vastaavantyyppisiä kokonaisuuksia.

5 Liitteet:

5.1 Alustavan testiohjelman ohjelmakoodi

```
int pushButton = 7;  // painonappi-muuttujan esittely ja alustus
int state = 0;   // painonapin tila -muuttujan esittely ja alustus

void setup() {

  pinMode(pushButton, INPUT);  // painonapin asetus inputiksi

  // initialize serial communication at 9600 bits per second:

  Serial.begin(9600);

    // initialize digital pin LED_BUILTIN as an output.

  pinMode(LED_BUILTIN, OUTPUT);

  pinMode(10, OUTPUT);

  pinMode(11, OUTPUT);

  pinMode(12, OUTPUT);

}

// the loop routine runs over and over again forever:

void loop() {

  // luetaan analogiatulot 0 ja 2 ja sijoitetaan ne sitten muuttujiin (lämpötila- ja kosteusmittaus)

  // (koska analogiatuloja, anturijännitteet menevät ensin 10 bittiselle AD-muuntimelle, josta tulevat arvot
välillä 0 - 1023)

  int sensorValue = analogRead(A0);
 int sensorValue2 = analogRead(A2);
float T = ((140.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T
```

```
  float RH = ((100 / 1023.0)*sensorValue2);
// kaava, jolla lasketaan suhteellinen kosteus, sijoitetaan tulos muuttujaan RH

  float V = (3.3 / 1023.0)*sensorValue ;
// kaava, jolla lasketaan lämpötila-anturin jännite, sijoitetaan tulos muuttujaan V

  float V2 = (3.3 / 1023.0)*sensorValue2 ;
// kaava, jolla lasketaan kosteusanturin jännite, sijoitetaan tulos muuttujaan V2

  state = digitalRead(pushButton); // luetaan painonapin tila

if ( state == 1 ){  // jos painonapin tila = 1 (painetaan nappia) mennään kosteusmittaukseen:

 Serial.println ("Suhteellinen huoneilman kosteus 25 asteessa C: "), Serial.println(RH), Serial.println ("%");

 Serial.println ("Absoluuttinen huoneilman kosteus 25 asteessa C: "), Serial.println(RH*22.8/100), Se-
rial.println ("g/m2");

 Serial.println ("**********");

 Serial.println ("Jännite: "), Serial.println(V2), Serial.println ("V");

 Serial.println ("**********");

   if (RH<30) {

 Serial.println ("On tosi kuivaa! punainen led palaa. ");

 Serial.println ("**********");

  digitalWrite(10, HIGH);   // turn the LED on (HIGH is the voltage level)

  digitalWrite(11, LOW);    // turn the LED off by making the voltage LOW

  digitalWrite(12, LOW);    // turn the LED off by making the voltage LOW

 // delay(100);            // wait for a moment

 // digitalWrite(11, LOW);  // turn the LED off by making the voltage LOW

 // delay(1000);           // wait for a moment

    }
```

```
  else if (RH>=30 && RH<70 ) {
Serial.println ("Aika sopivan kosteaa! Keltainen led palaa. ");
Serial.println ("**********");
 digitalWrite(11, HIGH);   // turn the LED on (HIGH is the voltage level)
  digitalWrite(10, LOW);    // turn the LED off by making the voltage LOW
 digitalWrite(12, LOW);    // turn the LED off by making the voltage LOW
// delay(100);                    // wait for a moment
// digitalWrite(11, LOW);    // turn the LED off by making the voltage LOW
// delay(1000);                    // wait for a moment
  }
  else {


Serial.println ("On tosi kosteaa! Sininen led palaa.");
Serial.println ("**********");
 digitalWrite(12, HIGH);   // turn the LED on (HIGH is the voltage level)
 digitalWrite(10, LOW);    // turn the LED off by making the voltage LOW
 digitalWrite(11, LOW);    // turn the LED off by making the voltage LOW
 // delay(100);                    // wait for a moment
// digitalWrite(12, LOW);    // turn the LED off by making the voltage LOW
// delay(1000);              // wait for a moment


  }
}
```

```
else { // jos painonapin tila = 0 (ei paineta nappia) mennään lämpötilanmittaukseen:
 Serial.println ("Lämpötila on "), Serial.println(T), Serial.println ("astetta C");
 Serial.println ("**********");


 Serial.println ("Jännite: "), Serial.println(V), Serial.println ("V");
 Serial.println ("**********");
  if (T<-10) {
 Serial.println ("hrr kun on kylymäääää!!");
 Serial.println ("**********");
  digitalWrite(12, HIGH);   // turn the LED on (HIGH is the voltage level)
  digitalWrite(10, LOW);    // turn the LED off by making the voltage LOW
  digitalWrite(11, LOW);    // turn the LED off by making the voltage LOW
    }
     else if (T>=-10 && T<0) {
       Serial.println ("ihan jees kuivaa talvisäätä!");
 Serial.println ("**********");
    digitalWrite(12, HIGH);   // turn the LED on (HIGH is the voltage level)
  digitalWrite(10, LOW);    // turn the LED off by making the voltage LOW
  digitalWrite(11, HIGH);    // turn the LED on (HIGH is the voltage level)
    }
        else if (T>=0 && T<17) {
      Serial.println ("Semmosta ok keväistä/syksyistä keliä.");
```

```
Serial.println ("**********");
  digitalWrite(12, LOW);   // turn the LED off by making the voltage
 digitalWrite(10, LOW);   // turn the LED off by making the voltage LOW
 digitalWrite(11, HIGH);   // turn the LED on (HIGH is the voltage level)
  }
        else if (T>=17 && T<25) {
    Serial.println ("Kesäistä säätä, muttei hellettäkään.");
Serial.println ("**********");
   digitalWrite(12, LOW);   // turn the LED off by making the voltage
 digitalWrite(10, HIGH);   // turn the LED on (HIGH is the voltage level)
 digitalWrite(11, HIGH);   // turn the LED on (HIGH is the voltage level)
  }
  else {
    Serial.println ("Helteestä polttavan kuumaan!");
Serial.println ("**********");
  digitalWrite(12, LOW);  // turn the LED off by making the voltage LOW
 digitalWrite(10, HIGH);   // turn the LED on (HIGH is the voltage level)
 digitalWrite(11, LOW);   // turn the LED off by making the voltage LOW
  }
 }
 delay(1000);       // delay in between reads for stability
}
```

5.2 Lopullinen ohjelmakoodi:

```
#include <ArduinoBLE.h>

BLEService measureService("16150f38-e7a9-4fe1-ae08-48464baf25b2");
// Luodaan BLE-palvelu "measureService"

BLEStringCharacteristic  temperatureLevel("ff99948c-18ff-4ed8-942e-512b9b24b6db", BLERead |
BLEWrite  |  BLENotify,15); // Alustetaan measureService lämpötilatiedon BLERead, BLEWrite ja BLENotify
-ominaisuuksilla

BLEStringCharacteristic  humidityLevel("ff99948c-18ff-4ed8-942e-512b9b24b6da", BLERead | BLEWrite  |
BLENotify,15);
//  Alustetaan measureService kosteustiedon BLERead, BLEWrite ja BLENotify -ominaisuuksilla

const int ledPin = LED_BUILTIN; //alustetaan sisäänrakennettu led (13)

// alustetaan muuttujat mittaustiedoille, jotka tullaan keskiarvoistamaan

        float T;
        float T0;
        float T1;
        float T2;
        float T3;
        float T4;
        float T5;
        float T6;
        float T7;
        float RH=0;
        float V=0;
        float V2=0;
        float v_humidity_sht30;




        void setup() {

          Serial.begin(115200); // alustetaan sarjaliikenneväylä
```

```
while (!Serial);

// asetetaan LEDit outputiksi

pinMode(ledPin, OUTPUT);
pinMode(2, OUTPUT);
pinMode(4, OUTPUT);
pinMode(6, OUTPUT);

// alustetaan BLE

if (!BLE.begin()) {

  Serial.println("starting BLE failed!");

  while (1);

}

// asetetaan Bluetooth palvelun mainostettava nimi ja itse palvelu mainostettavaksi:

BLE.setLocalName("Measurements");

BLE.setAdvertisedService(measureService);

//Asetetaan BLE palvelun ominaisuudet

measureService.addCharacteristic(temperatureLevel);

measureService.addCharacteristic(humidityLevel);



// lisätään BLE-palvelu

BLE.addService(measureService);




// asetetaan alkuarvot ominaisuuksille

  temperatureLevel.writeValue(String(0));
```

```
    humidityLevel.writeValue(String(0));


  // aloitetaan BLE-palvelun mainostaminen ja printataan sarjamonitoriin viesti.
  BLE.advertise();
  Serial.println("Bluetooth device active, waiting for connections...");
}


void loop() {


  BLEDevice central = BLE.central();
// BLE laite (Nano 33 iot) on valmiina ja odottaa lukijan liittymistä
  if (central) {
    Serial.print("Connected to central: "); // printataan sarjamonitoriin viesti lukijan liittymisestä.
     // printataan sarjamotnitoriin MAC-osoite
    Serial.println(central.address());
    // kun BLE-keskuslaite on liittyneenä etälaitteeseen...
   while (central.connected()) {
    // digitalWrite(ledPin, HIGH);
    // delay(50);
    // digitalWrite(ledPin, LOW);



     updateMeasureLevel();  // kutsutaan funktiota, jolla luetaan mittaustieto
    delay(1000);
```

```
  }

   digitalWrite(ledPin, LOW);  // sammutetaan led
  digitalWrite(2, LOW); //punainen LED
digitalWrite(4, LOW); // keltainen LED
digitalWrite(6, LOW); // sininen LED

   Serial.print("Disconnected from central: "); // kirjoitetaan viesti sarjamonitoriin

   Serial.println(central.address()); // kirjoitetaan keskuslaitteen osoite sarjamonitoriin

  }

}



// toteutetaan funktio, jolla luetaan mittaustieto

void updateMeasureLevel() {

int sensorValue = analogRead(A0);
// esitellään muuttuja johon luetaan  analogiatulo A0:n jännite

int sensorValueA = analogRead(A2);
//esitellään muuttuja johon luetaan  analogiatulo A2:n jännite

int sensorValueA3 = analogRead(A3);
//esitellään muuttuja johon luetaan  analogiatulo A3:n jännite

   digitalWrite(10, HIGH);

   delay(50);

   digitalWrite(10, LOW);




 T0 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T0
```

```
T1 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T1

T2 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T2

T3 = ((97.0 / 1023.0)*sensorValue)-40 ; // kaava, jolla lasketaan lämpötila, sijoitetaan tulos
muuttujaan T3

T4 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T4

T5 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T5

T6 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T6

T7 = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T7

T  = (T0+T1+T2+T3+T4+T5+T6+T7)/8;
// lasketaan T0...T7-muuttujien keskiarvo ja sijoitetaan se muuttujaan T ts.

delay(50);

// = tehdään lämpötilamittauksille keskiarvoistus,
jolla vähennetään mittausarvojen vaihteluväliä/kohinaa



// T = ((97.0 / 1023.0)*sensorValue)-40 ;
// kaava, jolla lasketaan lämpötila, sijoitetaan tulos muuttujaan T

// RH = ((100.0 / 1023.0)*sensorValueA3);
// kaava, jolla lasketaan suhteellinen kosteus, sijoitetaan tulos muuttujaan RH

V = (3.3 / 1023.0)*sensorValue ;
// kaava, jolla lasketaan lämpötila-anturin jännite, sijoitetaan tulos muuttujaan V

V2 = (3.3 / 1023.0)*sensorValueA3 ;
// kaava, jolla lasketaan kosteusanturin jännite, sijoitetaan tulos muuttujaan V2
```

```
  v_humidity_sht30 = (float)analogRead(A3) / 1024 * 3.3;

RH = -12.5 + 41.667 * v_humidity_sht30;
// kaava, jolla lasketaan suhteellinen kosteus, sijoitetaan tulos muuttujaan RH

 temperatureLevel.writeValue(String(T));
//kirjoitetaan T:n arvo BLE palvelun  temperatureLevel -ominaisuusmuuttujaan

 humidityLevel.writeValue(String(RH));
 //kirjoitetaan RH:n arvo BLE palvelun  humidityLevel -ominaisuusmuuttujaan

 delay(50);



Serial.print("Lämpötila on: "), Serial.print(T);

Serial.print('\n');

Serial.print("Jännite on: "), Serial.print(V);

Serial.print('\n');

Serial.print("Suhteellinen kosteusproentti on: "), Serial.print(RH);

Serial.print('\n');

Serial.print("Jännite on: "), Serial.print(V2);

Serial.print('\n');



if (T < 0) {

  digitalWrite(2, LOW); //punainen LED

  digitalWrite(4, LOW); // keltainen LED

  digitalWrite(6, HIGH); // sininen LED

  }
```

```
else if (T >= 0 && T < 10) {
 digitalWrite(2, LOW); //punainen LED
 digitalWrite(4, HIGH); // keltainen LED
 digitalWrite(6, HIGH); // sininen LED
 }

else if (T >= 10 && T < 20) {
 digitalWrite(2, LOW); //punainen LED
 digitalWrite(4, HIGH); // keltainen LED
 digitalWrite(6, LOW); // sininen LED
 }

else if (T >= 20 && T < 38) {
 digitalWrite(2, HIGH); //punainen LED
 digitalWrite(4, HIGH); // keltainen LED
 digitalWrite(6, LOW); // sininen LED
 }

  else {
 digitalWrite(2, HIGH); //punainen LED
 digitalWrite(4, LOW); // keltainen LED
 digitalWrite(6, LOW); // sininen LED
 }
```

```
if (RH >= 0 && RH < 35 ) {
digitalWrite(ledPin, HIGH); //vihreä LED
delay(30);
digitalWrite(ledPin, LOW); //vihreä LED

}

else  if (RH >= 35 && RH < 55 ) {
digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
digitalWrite(ledPin, LOW); //vihreä LED

}

  else  if (RH >= 55 && RH < 65 ) {
digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
digitalWrite(ledPin, LOW); //vihreä LED
delay(80);
digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
```

```
digitalWrite(ledPin, LOW); //vihreä LED


}


  else  if (RH >= 65 && RH < 75 ) {
digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
digitalWrite(ledPin, LOW); //vihreä LED
delay(80);
digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
digitalWrite(ledPin, LOW); //vihreä LED
delay(80);
digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
digitalWrite(ledPin, LOW); //vihreä LED
}
   else if (RH >= 75 && RH < 90 ) {
 digitalWrite(ledPin, HIGH); //vihreä LED
delay(60);
digitalWrite(ledPin, LOW); //vihreä LED
delay(80);
digitalWrite(ledPin, HIGH); //vihreä LED
```

```
delay(60);

digitalWrite(ledPin, LOW); //vihreä LED

delay(80);

digitalWrite(ledPin, HIGH); //vihreä LED

delay(60);

digitalWrite(ledPin, LOW); //vihreä LED

delay(80);

digitalWrite(ledPin, HIGH); //vihreä LED

delay(60);

digitalWrite(ledPin, LOW); //vihreä LED

}

  else {

 digitalWrite(ledPin, HIGH); //vihreä LED

}

}
```